

Break the login

Raport scris

Nichita Iulia-Nicoleta, 341

Aprilie 2026

1 Introducere

Compania fictivă *AuthX* utilizează o aplicație web pentru a gestiona accesul la resurse sensibile al angajaților săi. Pentru a îndeplini această condiție, aplicația salvează datele utilizatorilor (Users), biletele stocate (Tickets) și monitorizează activitățile (Audit logs).

Un utilizator este recunoscut din momentul înregistrării în platformă prin *formularul de register*, iar apoi cu aceste date se poate loga prin *formularul de login*. Utilizatorii sunt înregistrați cu rolul fiecăruia, ANALYST sau MANAGER. Rolul de ANALYST permite vizualizarea și editarea propriilor bilete, precum și editarea profilului. Rolul de MANAGER are aceleași permisiuni ca cele de ANALYST la care se adaugă și vizualizarea activităților înregistrate. Indiferent de rol, utilizatorilor le este permis să își reseteze parola contului personal prin *formularul de resetpassword*, adăugând email-ul contului și adăugând noua parolă pe o pagină cu acest scop.

Arhitectură

Aplicația AuthX utilizează o arhitectură client-server clasică, în care frontend-ul comunică cu backend-ul printr-un API REST. Frontend-ul aplicației este compus din pagini *HTML statice*, stilizate cu CSS, care utilizează formulare pentru a primi datele introduse de utilizatori în câmpurile dedicate. Schimbul de date dintre interfață și server se realizează prin logica de fetch, utilizând formatul JSON și metodele standard HTTP: GET, POST, PUT și DELETE. S-a ales HTML static pentru simplitate și timp de încărcare redus, fără overhead de framework.

Backend-ul este implementat prin framework-ul *Express*. Are rolul de a procesa cererile primite, de a realiza logica funcționalităților platformei și de a gestiona accesul la baza de date prin metodele enumerate anterior. Acesta expune un set de endpoint-uri REST prin care frontend-ul poate efectua operații asupra entităților aplicației: users, tickets, audit_logs. Baza de date aleasă este *SQLite*, datorită simplității, portabilității și performanței, fiind o

bază de date relațională ce permite utilizarea metodelor de tip CRUD și o soluție de tip *zero-configuration*," stocată într-un singur fișier, fără a necesita un server dedicat.

Aplicația adoptă practici de securitate cibernetică pentru protecția împotriva XSS(Cross-Site Scripting), CSRF(Cross-Site Request Forgery), pentru prevenirea injection sau hijacking. Au fost alese în acest sens: JWT (JSON Web Token implementat prin librăria `jsonwebtoken`) trimis printr-un cookie de sesiune, bcrypt, crypto (build-in Node.js module).

2 Setup mediu

Această secțiune descrie pașii necesari pentru pregătirea mediului de lucru și instalarea dependențelor software.

2.1 Virtualizare și mediu de lucru

Pentru dezvoltarea aplicației s-au utilizat:

- Sistem de operare folosit: Windows 11
- Node.js: v24.11.0
- npm: 11.6.1
- Bază de date utilizată: SQLite (sqlite3 library - 3.46.1 (64-bit))
- Editor: VS Code

Utilizarea unei mașini virtuale (VirtualMachine) este justificată prin beneficiile tehnice aduse aplicației AuthX. S-a utilizat Oracle VirtualBox (Version 7.1.2) pentru protejarea sistemului gazdă (host) de eventuale vulnerabilități testate în timpul dezvoltării, asigurarea că aplicația rulează pe o configurație „curată”, fără conflicte cu alte programe instalate pe calculatorul personal.

Configurația Hardware VirtualMachine:

- Procesor (vCPU): 5 nuclee
- Memorie RAM: 16384 MB
- Stocare: 50GB (dinamic alocată)
- Rețea: NAT (pentru acces la internet)

2.2 Instalare și utilizare a framework-ului și a bazei de date

Express este un framework pentru dezvoltarea aplicațiilor web ce utilizează *Node.js* și oferă acces la funcționalități esențiale, intuitiv create, ușor de utilizat pentru a ajuta dezvoltatorii să creeze aplicații ce demonstrează scalabilitate și performanță ridicată.

Configurație 1: Instalare Node.js și Express

```
# Instalare Node.js
sudo apt update
sudo apt install nodejs npm      # Linux

# Verificare versiuni
node -v
npm -v

# Creeaza package.json*
npm init

npm install express
```

*package.json este creat pentru identificarea proiectului și gestionarea dependențelor

Pentru a asigura o structură modulară, proiectul valorifică funcționalități ale framework-ului Express, facilitând astfel implementarea unui setup minimalist cu metodele disponibile (ex: `express()`, `app.listen()`), Express fiind considerat un *Minimalist framework*.

Acest framework pune la dispoziție un *sistem de rutare* ce permite definirea rutelor pentru metodele HTTP cunoscute (GET, POST, PUT, DELETE), ceea ce aduce flexibilitate aplicației dezvoltate.

Cod Sursă 1: Definire rute, punerea în funcțiune a server-ului

```
import express from 'express';

const app = express();
const port = 3000;

app.use(express.json()); /*

app.get('/login', (req, res) => {
  res.sendFile(path.join(__dirname, './templates/login.html'));
});
// login.html - HTML static ce afiseaza formularul de login

// ...
```

```
app.listen(port, () => {
  console.log('App listening at http://localhost:${port}');
});
```

* `express.json()` definește un middleware definit de Express ce permite server-ului să par-seze cereri cu payload în format JSON.

Deoarece Express utilizează mediu de execuție Node.js, aplicația poate gestiona *cereri asincrone*. Acest aspect permite server-ului să proceseze mai multe cereri simultan, ceea ce îmbunătățește performanța sistemului.

Gestionarea răspunsurilor la cererile primite este un proces important în aplicații web și implicit *tratarea erorilor*. Platforma utilizează o abordare de tratare a erorilor pe mai multe niveluri. Strategia se concentrează pe prevenirea stărilor de "crash" ale serverului și pe furnizarea de coduri de stare corespunzătoare.

Înainte de interacțiunea cu baza de date, aplicația verifică integritatea datelor primite în corpul cererii (`req.body`). Pentru a asigura consistența și a evita duplicarea codului, a fost implementată funcția helper `badRequest` (vezi în *Cod sursă 2* și *Cod sursă 3*), care returnează un cod de stare **400 Bad Request**. Interacțiunile cu baza de date SQLite sunt asincrone și utilizează modelul error-first callback (Vezi *Cod sursă 2*). Deoarece biblioteca `jsonwebtoken` aruncă excepții în mod sincron atunci când un token este invalid sau expirat, procesul de verificare (`jwt.verify`) este încapsulat într-un bloc `try-catch` (vezi în *Cod sursă 4*). Acest lucru previne oprirea serverului și permite returnarea unui răspuns de tip **401 Unauthorized**.

Cod Sursă 2: Cerere de tip POST, tratare erori (fișier `'/api/user'`)

```
import express from 'express';

const router = express.Router();

// *
function badRequest(res, message) {
  return res.status(400).json({ error: message });
}

// creare user nou
// cerere de tip POST
router.post('/register', (req, res) => {
  const { email, password, role, locked } = req.body;

  // ... (logica validare input)

  const sql = // ... (interogare catre baza de date SQLite)
```

```

    // rularea interogarii pentru salvarea user-ului
    db.run(sql, [email, password, role, normalizedLocked],
function (err) {
    // tratare erori**
    // error-first callback
    if (err) return res.status(500).json({ error: err.message
});

    // ... (logica salvare actiune in audit_logs)

    // 201 successful HTTP request
    res.status(201).json({
        id: this.lastID,
        email,
        role,
        // ...
    });
});
});

export default router;

```

*în cazul datelor neconforme introduse de utilizator la completarea formularului, este returnat un răspuns cu statutul 400, considerat Client Error (datele introduse de client nu corespund)

erorile survenite în timpul interogării bazei de date sunt tratate local, returnând imediat un răspuns cu statusul **500 Internal Server Error, iar codul verifică prezența obiectului de eroare înainte de a procesa rezultatele, asigurând astfel integritatea fluxului de date

Cod Sursă 3: Definirea rutei pentru gestionarea cererilor API

```
app.use('/api/user', userRoutes);
```

Cod Sursă 4: Cerere de tip PUT, tratare erori în bloc `try-catch`

```

// Update current user profile
// cerere de tip PUT
router.put('/profile', (req, res) => {
    const token = req.cookies.auth_token;

    if (!token) return res.status(401).json({ error: 'Login
required' });

```

```

let decoded;
try {
  // extragerea payload-ului
  decoded = jwt.verify(token, 'abc');
} catch (err) {
  // 401 Unauthorized
  return res.status(401).json({ error: 'Invalid token' });
}

const { email } = req.body;

if (!email) {
  return badRequest(res, 'email cannot be empty');
}

// rulare interogare baza de date
db.run('UPDATE users SET email = ? WHERE id = ?', [email,
decoded.userId], function (err) {
  if (err) return res.status(500).json({ error: err.message
});
  if (this.changes === 0) return res.status().json({ error:
'User not found' });

  // ... (logica salvare actiune in audit_logs)

  res.json({ updatedID: decoded.userId });
});
});

```

Aplicația AuthX returnează și alte coduri de stare, precum **403 forbidden**. Unul dintre contextele în care este declanșată este încercarea unui user cu rol ANALYST de a șterge un cont care nu este al său (Vezi *Cod sursă 5*). Aplicația returnează și **404 Not found**, declanșată la apelul endpoint-ului `/api/user/login` la trimiterea unui input de email care nu există (`error: 'User not found'`) (Vezi *Cod sursă 6*).

Cod Sursă 5: Cerere de tip PUT, tratare erori în bloc `try-catch`

```

// Delete user
// cerere de tip PUT
router.delete('/:id', (req, res) => {
  const userId = Number(req.params.id);

  const token = req.cookies.auth_token;

```

```

    if (!token) return res.status(401).json({ error: 'Login
required' });

    try {
        const decoded = jwt.verify(token, 'abc');

        if (decoded.userId !== userId) {
            // 403 Forbidden
            return res.status(403).json({ error: 'You can only
delete your own account' });
        }

        db.run('DELETE FROM users WHERE id = ?', [userId],
function (err) {
    if (err) return res.status(500).json({ error: err.
message });
    if (this.changes === 0) return res.status(404).json({
error: 'User not found' });

    // ... (logica salvare actiune in audit_logs)

    res.json({ deletedID: userId });
});
} catch (err) {
    res.status(401).json({ error: 'Invalid token' });
}
});

```

Cod Sursă 6: Cerere de tip POST, implementare logică login

```

// Login
// cerere de tip POST
router.post('/login', (req, res) => {
    const { email, password_hash } = req.body;

    // interogare catre baza de date pentru determinarea user-ului
    db.get('SELECT * FROM users WHERE email = ?', [email], async (
err, user) => {
        if (err) return res.status(500).json({ error: err.message
});
        if (!user) {
            // ... (logica salvare actiune in audit_logs)

            // 404 Not found
            return res.status(404).json({ error: 'User not found'
});
        }
    });
});

```

```

});
    }

    // ... (logica validare parola)

    // parola este corecta
    if (match) {
        // ... logica creeare token

        // ... (logica salvare token in cookie)

        // ... (logica salvare actiune in audit_logs)

        res.json({ success: true, message: "Authentication
successful" });
    } else {
        // ... (logica salvare actiune in audit_logs)

        res.status(401).json({ error: 'Incorrect password' });
    }
});
});

```

Pentru gestionarea persistentei datelor, proiectul utilizează **SQLite**, un motor de bază de date relațional de tip embedded. Integrarea în mediul Node.js a fost realizată prin intermediul bibliotecii `sqlite3`, utilizând interogări parametrizate pentru a preveni atacurile de tip SQL Injection. Pachetul `sqlite3` oferă driverul necesar pentru ca codul Javascript să poată comunica cu fișierul bazei de date.

Configurație 2: Instalare librărie

```
npm install sqlite3
```

Următorul pas este crearea unei conexiuni către fișierul SQLite (`'data.sqlite'`). Dacă acesta nu există, SQLite îl va crea automat. Acest fișier este de fapt un sistem de fișiere structurat extrem de complex ce conține toată baza de date definită.

Pentru aplicația AuthX, baza de date stochează datele în 3 entități: users, tickets, audit_logs.

Entitatea users are următoarele câmpuri: id, email, password_hash, role, reset_token, reset_token_expired_at, created_at, locked. Sunt stocate date despre conturile utilizato-

rilor, `reset_token`, precum și `reset_token_expired_at` sunt utilizate la funcționalitatea de resetare a parolei contului personal, iar `locked` este utilizat pentru a identifica un cont blocat, din cauza încercărilor multiple de logare (credential stuffing / brute-force).

Entitatea `tickets` are următoarele câmpuri: `id`, `title`, `description`, `severity`, `status`, `owner_id`, `created_at`, `updated_at`. Sunt stocate edate despre biletele existente în aplicație, iar câmpul `owner_id` este cheie străină către entitatea `users`, câmpul `users.id` pentru a identifica cine este deținătorul biletului.

Entitatea `audit_logs` are următoarele câmpuri: `id`, `user_id`, `action`, `resource`, `resource_id`, `timestamp`, `ip_address`. Sunt stocate date despre activitățile din platformă, precum: crearea unui cont nou, logarea într-un cont existent, resetarea parolei, adăugarea unui nou bilet (de către manager). Câmpul `user_id` este cheie străină către entitatea `users`, câmpul `users.id` pentru a determina ce user a realizat acțiunea, câmpul `resource` poate fi `USERS` sau `TICKETS` pentru a detemrina asupra cui a avut loc acțiunea, iar câmpul `resource_id` este utilizat pentru a reține id-ul resursei asupra căreia s-a realizat acțiunea.

Cod Sursă 7: Creare conexiune către baza de date și crearea tabelor

```
import sqlite3 from 'sqlite3';

const dbPath = './data.sqlite';
const db = new sqlite3.Database(dbPath, (err) => {
  if (err) console.error(err.message);

  console.log('Connected to SQLite.');
```

});

db.serialize(() => {
 // fara aceasta comanda se pot introduce id-uri care nu exista
 db.run('PRAGMA foreign_keys = ON', (err) => {
 if (err) console.error("Failed to enable FK support:", err
.message);
 });

 db.run('CREATE TABLE IF NOT EXISTS users (
 id INTEGER PRIMARY KEY AUTOINCREMENT,
 email TEXT UNIQUE NOT NULL,
 password_hash TEXT NOT NULL,
 role TEXT CHECK(role IN ('ANALYST', 'MANAGER')) NOT NULL,
 created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
 reset_token TEXT,
 reset_token_expires_at DATETIME,
 locked INTEGER DEFAULT 00 CHECK(locked IN (0, 1))
)', (err) => { if (err) console.error('users:', err.message);
});

 db.run('CREATE TABLE IF NOT EXISTS tickets (

```

        id INTEGER PRIMARY KEY AUTOINCREMENT,
        title TEXT NOT NULL,
        description TEXT,
        severity TEXT CHECK(severity IN ('LOW', 'MED', 'HIGH'))
NOT NULL,
        status TEXT CHECK(status IN ('OPEN', 'IN_PROGRESS', '
RESOLVED')) NOT NULL,
        owner_id INTEGER,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        FOREIGN KEY (owner_id) REFERENCES users(id)
            ON DELETE CASCADE
            ON UPDATE CASCADE
    ), (err) => { if (err) console.error('tickets:', err.message)
; });

db.run('CREATE TABLE IF NOT EXISTS audit_logs (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER,
    action TEXT NOT NULL,
    resource TEXT,
    resource_id TEXT,
    timestamp DATETIME DEFAULT CURRENT_TIMESTAMP,
    ip_address TEXT,
    FOREIGN KEY (user_id) REFERENCES users(id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
    ), (err) => { if (err) console.error('audit_logs:', err.
message); });

});

export default db;

```

SQLite folosește metode diferite în funcție de tipul de rezultat așteptat:

- `db.run()`: Pentru acțiuni care nu returnează date (INSERT, UPDATE, DELETE).
- `db.get()`: Pentru a prelua un singur rând (ex: la Login).
- `db.all()`: Pentru a prelua toate rândurile dintr-o listă (ex: /assignees).

Cod Sursă 7: Interogări către baza de date SQLite

```
// .GET
```

```

db.get('SELECT * FROM users WHERE email = ?', [email], async (err,
    user) => {
    if (err) return res.status(500).json({ error: err.message
    });
    if (!user) {
        // ... (logica salvare actiune in audit_logs) (Vezi
        Cod sursa 8)

        return res.status(404).json({ error: 'User not found'
    });
    }

    // ... (logica validare parola)

    // daca parola este corecta
    if (match) {
        // ... (logica login cu succes)
    }
});

// .RUN
db.run('UPDATE users SET email = ? WHERE id = ?', [email, decoded.
    userId], function (err) {
    if (err) return res.status(500).json({ error: err.message });
    if (this.changes === 0) return res.status(404).json({ error: '
    User not found' });

    // ... (logica salvare actiune in audit_logs) (Vezi Cod sursa
    8)

    res.json({ updatedID: decoded.userId });
});

// .ALL
db.all('SELECT id, email, role FROM users ORDER BY email ASC', [],
    (err, users) => {
    if (err) return res.status(500).json({ error: err.message });

    // ... (logica returnare raspuns)
});

```

Pentru salvarea acțiunilor în tabelul audit_logs au fost create 2 funcții suplimentare pentru determinarea adresei ip a browser-ului care a făcut cererea, precum și salvarea în baza de date, popularea câmpurilor înregistrării cu date corespunzătoare.

Cod Sursă 8: Funcții suplimentare pentru noi înregistrări în entitatea audit_logs

```
import db from '../database.js';

function getClientIp(req) {
  // 'x-forwarded-for' contine adresa ip a cererii
  const forwarded = req.headers['x-forwarded-for'];

  if (typeof forwarded === 'string' && forwarded.trim() !== '')
  {
    return forwarded.split(',')[0].trim();
  }

  const ip = req.ip ?? null;

  // converteste ::1 la 127.0.0.1
  if (ip === '::1') return '127.0.0.1';

  // converteste ::ffff:192.168.1.1 la 192.168.1.1
  if (ip?.startsWith('::ffff:')) return ip.substring(7);

  return ip;
}

export function logAudit({ req, userId = null, action, resource =
  null, resourceId = null }) {
  if (!action) {
    return;
  }

  const resourceIdValue = resourceId === null || resourceId ===
  undefined
    ? null
    : String(resourceId);

  // inserare in baza de date
  db.run(
    `
      INSERT INTO audit_logs (user_id, action, resource,
      resource_id, ip_address)
      VALUES (?, ?, ?, ?, ?)
    `,
    [userId, action, resource, resourceIdValue, getClientIp(
  req)],
    (err) => {
      if (err) {
        console.error('Audit log insert failed:', err.
  message);
      }
    }
  );
}
```

```
    }  
  }  
);  
}
```

Pentru vizualizarea mai ușoară a entităților bazei de date este utilizat DB Browser for SQLite Version 3.13.1.

3 Implementare MVP

Sistem de autentificare

Aplicația AuthX implementează un sistem de autentificare ce include logica de înregistrare în platformă (register), precum și logica de logare într-un cont existent (login). Sistemul utilizează sesiuni (biblioteca `express-session`) și mecanisme de stocare securizată prin **HTTP Cookies**, precum și biblioteca `bcrypt` pentru hashuirea parolelor stocate și modulul `crypto` pentru a crea un token aleator în contextul resetării parolei. Procesul de autentificare urmează o logică strictă pentru a preveni accesul neautorizat:

1. Register:

- utilizatorul completează un formular de creare cont cu email, password și role (ANALYST sau MANAGER)
- datele de input sunt validate în backend, la nivelul cererii `/api/user/register`
- dacă toate datele sunt valide (ex: password de o lungime mai mare de 8 caractere ce conține cel puțin o literă mare, o literă mică, un număr și un caracter special, etc.) sunt stocate sub forma unei înregistrări în tabelul users, cu input-ului din câmpul password hash-uit (`bcrypt.hash`)
- dacă un câmp nu trece de pasul de validare, utilizatorul primește un mesaj cu requirement-ul necesar

2. Login:

- utilizatorul completează un formular cu email și password
- se verifică dacă există un utilizator cu email-ul respectiv în baza de date
- dacă există, se hash-uiește parola (password) pentru a fi comparată cu cea stocată în baza de date(`bcrypt.compare`)
 - dacă parolele corespund se creează sesiunea cu datele necesare pentru verificarea identității în cererile ulterioare (Vezi *Cod sursă*)

- dacă parolele nu corespund, crește cu 1 unitate valoarea stocată în câmpul `users.failed_login_attempts` pentru ca la un număr N (considerat 5) contul să fie blocat, acțiunea fiind suspectă pentru un atac de tip Brute-Force
 - dacă nu există, este afișat un mesaj generic pentru a nu oferi informații suplimentare atacatorilor (email-ul respectiv nu este în baza de date în acest context)
3. **Logout:** fiecare utilizator are pe pagina propriului profil un buton de logout care face ca sesiunea să nu mai fie validă (`req.session.destroy`) și șterge cookie-ul de sesiune (`res.clearCookie`)
 4. Resetare parolă
 - utilizatorul accesează pagina `/forgotpassword` și își adaugă email-ul într-un formular
 - dacă email-ul există este generat un token aleator care este stocat în câmpul `users.reset_token` și este accesat un URL de forma `/resetpassword/:token` (unde `token = users.reset_token`), această pagină conține un formular în care utilizatorul poate adăuga noua parolă ce va fi salvată în baza de date
 - dacă email-ul nu există este afișat un mesaj generic
 5. **Gestionare sesiuni:** menținerea autentificării între cereri se face prin datele de sesiune ce salvează id-ul utilizatorului curent precum și un câmp manager pentru a determina eficient dacă are autorizare, iar la fiecare cerere aceste date sunt verificate prin intermediul unui middleware creat în acest sens(`authMiddleware` returnează datele contului sesiunii curente) (Vezi *Cod sură*)

CRUD

Sistemul implementează modelul de persistență CRUD pentru gestionarea ciclului de viață al resurselor (ex: users, tickets, audit_logs), asigurând integritatea și manipularea corectă a datelor prin intermediul API-ului.

În cazul tabelii **users**, operațiile au fost implementate astfel:

Operație	Rută (Endpoint)	Metodă	Descriere Funcțională
CREATE	/api/user/register	POST	Înregistrarea unui cont nou și persistarea hash-ului parolei în baza de date.
CREATE	/api/user/login	POST	Logarea într-un cont existent.
READ	/api/user/profile	GET	Extragerea profilului utilizatorului (fără date sensibile) pe baza ID-ului unic.
UPDATE	/api/user/profile	PUT	Actualizarea informațiilor de profil (<i>email</i> sau <i>password</i>).
DELETE	/api/user	DELETE	Eliminarea definitivă a înregistrării utilizatorului și a sesiunilor active.

În cazul tablei **tickets**, operațiile au fost implementate astfel:

Operație	Rută (Endpoint)	Metodă	Descriere Funcțională
CREATE	/api/tickets	POST	Crearea unui nou ticket cu input-ul din formular.
READ	/api/tickets	GET	Extragerea datelor despre tickets și afișarea corespunzătoare în funcție de permisiuni (MANAGER poate vizualiza toate tickets, iar ANALYST doar pe cele proprii).
UPDATE	/api/tickets/:id	PUT	Actualizarea informațiilor despre un ticket identificat prin id.
DELETE	/api/tickets/:id	DELETE	Eliminarea definitivă a înregistrării ticket-ului din baza de date.

În cazul tablei **audit_logs**, în logica aplicației au fost necesare doar operațiile de READ și CREATE:

Operație	Rută (Endpoint)	Metodă	Descriere Funcțională
CREATE	-	-	Înregistrarea unui nou activități când este cazul printr-o funcție suplimentară (Vezi <i>Cod sursă 8</i>).
READ	/api/auditlogs	GET	Extragerea înregistrărilor din tabela audit_logs.

Search

Modulul de căutare permite utilizatorului să navigheze printre înregistrările tickets filtrate după *status*, *severity* și doar după *title* în cazul ANALYST-ului, și după *title* și a *email-ul utilizatorului* care îl deține (prin câmpul `tickets.owner_id` se determină prin intermediul instrucțiunii JOIN email-ul utilizatorului) în cazul MANAGER-ului.

Pe pagina profilului personal, utilizatorul are 3 casete de filtrare: search pentru introducerea input-ului de text pentru a filtra după titlu și email, un element de tip select pentru a alege valoarea de filtrare pentru status și un element de tip select pentru a alege valoarea de filtrare pentru severity. Valorile de filtrare sunt extrase în pagina *profile.html*.

Cod Sursă 9: Definire elemente frontend pentru filtrare

```
<div class="tickets-tools">
  <div class="ticket-filters" id="ticketFilters">
    <input
      id="ticketSearch"
      class="ticket-search"
      type="search"
      placeholder="Search after title"
      aria-label="Search tickets"
      oninput="applyTicketFilters()"
    >
    <select id="statusFilter" onchange="applyTicketFilters()" aria-label="Filter by status">
      <option value="">All statuses</option>
      <option value="OPEN">Open</option>
      <option value="IN_PROGRESS">In progress</option>
      <option value="RESOLVED">Resolved</option>
    </select>
    <select id="severityFilter" onchange="applyTicketFilters()" aria-label="Filter by severity">
      <option value="">All severities</option>
      <option value="LOW">Low</option>
      <option value="MED">Medium</option>
      <option value="HIGH">High</option>
    </select>
    <button type="button" class="filter-reset-btn" onclick="resetTicketFilters()">Clear</button>
  </div>
</div>
```

Gestiunea filtrelor și a căutării

Sistemul permite filtrarea dinamică a tickets prin 3 funcții principale care gestionează starea interfeței și comunicarea cu API-ul.

- **Construirea query-ului:** sunt extrase datele din câmpul de căutare, filtrul de status și filtrul de severity, se utilizează obiectul nativ `URLSearchParams()` pentru a construi șirul de interogare ce asigură faptul că caracterele speciale din bara de căutare sunt encodeate corect

Cod Sursă 10: Construire query

```
function buildTicketFilterQuery() {
    const searchInput = document.getElementById('ticketSearch');
    const search = searchInput ? searchInput.value.trim() : '';
    const status = document.getElementById('statusFilter').value;
    const severity = document.getElementById('severityFilter').value;
    const query = new URLSearchParams();

    if (search) {
        query.set('search', search);
    }
    if (status) {
        query.set('status', status);
    }
    if (severity) {
        query.set('severity', severity);
    }

    const queryString = query.toString();
    return queryString ? `?${queryString}` : '';
}
```

- **Aplicarea și resetarea filtrelor:** `applyTicketFilters` declanșează reîncărcarea datelor, iar `resetTicketFilters` le aduce la starea inițială - o funcție ce permite utilizatorului să reseteze toate filtrele într-un singur click

Cod Sursă 11: Funcții din spatele butoanelor

```

function applyTicketFilters() {
    loadTickets();
}

function resetTicketFilters() {
    const searchInput = document.getElementById('
ticketSearch');
    if (searchInput) {
        searchInput.value = '';
    }
    document.getElementById('statusFilter').value = '';
    document.getElementById('severityFilter').value = '';
    loadTickets();
}

```

- **Sincronizarea cu serverul:** `loadTickets` utilizează API-ul `fetch` pentru a efectua o cerere asincronă către backend, endpoint-ul apelat se schimbă în funcție de filtrele aplicate (ex: `/api/tickets` vs `/api/tickets/status=OPEN`)

Cod Sursă 12: Cerere asincronă către backend

```

async function loadTickets() {
    try {
        const response = await fetch(`/api/tickets${
buildTicketFilterQuery()}`);
        const tickets = await response.json();

        const tbody = document.getElementById('ticketsBody
');
        tbody.innerHTML = '';

        // ... (logica gestionare erori)

        // ... (logica construire elemente html)
    } catch (error) {
        console.error("Tickets error:", error);
    }
}

```

Această abordare de tip "Stateless search" este benefică, deoarece permite utilizatorului

să distribuie URL-ul cu filtrele de search și să rezulte aceeași listă filtrată de tickets.

Endpoint-ul de tip GET implementează o logică de filtrare dinamică a tickets, adaptându-se în funcție de permisiunile utilizatorului și parametrii primiți. Înainte de execuție serverul verifică opțiunile de status și severity pentru a se încadra în valorile permise, Interogarea SQL nu este fixă, ea se construiește pe parcurs, adăugând condiții doar pentru filtrele care au fost completate în interfață.

Cod Sursă 13: Validare date de filtrare

```
const VALID_SEVERITIES = new Set(['LOW', 'MED', 'HIGH']);
const VALID_STATUSES = new Set(['OPEN', 'IN_PROGRESS', 'RESOLVED',
  ]);

const statusFilterRaw = typeof req.query.status === 'string' ? req
  .query.status.trim() : '';
const severityFilterRaw = typeof req.query.severity === '
string' ? req.query.severity.trim() : '';
const searchFilterRaw = typeof req.query.search === 'string' ?
  req.query.search.trim() : '';

if (statusFilter && !VALID_STATUSES.has(statusFilter)) {
  return badRequest(res, 'status must be OPEN, IN_PROGRESS
or RESOLVED');
}
if (severityFilter && !VALID_SEVERITIES.has(severityFilter)) {
  return badRequest(res, 'severity must be LOW, MED or HIGH'
);
}
```

Cod Sursă 14: Interogare SQL flexibilă

```
let sql = 'SELECT * FROM tickets WHERE owner_id = ?';
const queryParams = [req.user.id]; // id-ul user-ului preluat din
  sesiune

if (statusFilter) {
  sql += ' AND status = ?';
  queryParams.push(statusFilter);
}
if (severityFilter) {
  sql += ' AND severity = ?';
  queryParams.push(severityFilter);
}
```

```

if (searchFilter) {
  sql += ' AND LOWER(title) LIKE ?';
  queryParams.push('%${searchFilter.toLowerCase()}%');
}

sql += ' ORDER BY created_at DESC';

db.all(sql, queryParams, (err, tickets) => {
  if (err) return res.status(500).json({ error: err.message });
  res.json(tickets);
});

```

4 Prezentare vulnerabilități

Aplicația AuthX este construită având în vedere vulnerabilitățile la care este expusă o aplicație web astfel încât riscurile să fie minimizate.

Cross-Site Scripting (XSS) este un atac ce se bazează pe injection, ce presupune script-uri malițioase introduse în site-uri web de încredere. Această vulnerabilitate apare atunci când aplicația include date furnizate de utilizator în paginile generate, fără a le valida sau encoda în prealabil. Browserul victimei nu poate distinge între scriptul legitim și cel atacator, executându-l automat sub autoritatea site-ului vizitat. Consecințele includ furtul modulelor de sesiune (cookies), compromiterea conturilor și manipularea conținutului afișat utilizatorului final.

SQL Injection reprezintă o vulnerabilitate de securitate critică în care un atacator "injectează" comenzi SQL malițioase în câmpurile de input ale unei aplicații. Scopul este de a păcăli aplicația să execute instrucțiuni pe care dezvoltatorul nu le-a prevăzut, oferind acces neautorizat la baza de date din spatele serverului. Atacul profită de lipsa filtrării datelor introduse de utilizator, permițând manipularea interogărilor pentru a extrage informații confidențiale (ex: liste de utilizatori sau parole), pentru a modifica înregistrări sau chiar pentru a șterge baze de date întregi.

Cross-Site Request Forgery (CSRF) este un atac ce forțează un utilizator autentificat să execute acțiuni nedorite într-o aplicație web fără consimțământul său. Atacul profită de faptul că browserele includ automat acreditările (precum cookie-urile de sesiune) în cererile către un site. De regulă, țintele sunt acțiunile care schimbă starea datelor, cum ar fi modificarea parolei, a adresei de email sau efectuarea de tranzacții financiare.

Prevenirea eficientă nu se bazează pe restricționarea metodelor HTTP (POST) sau pe HTTPS, ci pe utilizarea token-urilor anti-CSRF unice pentru fiecare sesiune. O altă metodă modernă de protecție este setarea atributului SameSite pe Strict sau Lax pentru cookie-uri, restricționând trimiterea acestora în contexte cross-site.

Insecure Direct Object Reference (IDOR) reprezintă o vulnerabilitate critică de

control a accesului ce apare atunci când o aplicație web permite utilizatorilor să vizualizeze sau să modifice date prin simpla manipulare a identificatorilor din URL sau din corpul cererilor. Prevenirea acestei breșe necesită o arhitectură care verifică drepturile de proprietate înaintea oricărei operații asupra bazei de date, eliminând încrederea implicită în parametrii primiți de la client.

Session Hijacking reprezintă preluarea neautorizată a unei sesiuni active prin furtul sau ghicirea codului de identificare (session token) pe care serverul îl folosește pentru a recunoaște un utilizator logat. Odată ce atacatorul obține acest token prin metode precum interceptarea traficului (sniffing) sau atacuri de tip XSS, el poate accesa contul victimei fără a cunoaște parola. Protecția principală constă în utilizarea protocolului HTTPS, generarea unor identificatori complexi și setarea atributelor HttpOnly și Secure pe cookie-uri, măsuri care împiedică accesul scripturilor malițioase la datele de sesiune.

Brute Force este un atac prin care un software automatizat încearcă sistematic mii de combinații de parole sau liste de cuvinte până când găsește varianta corectă. Atacatorii folosesc această metodă pentru a sparge conturi de utilizator sau pentru a descoperi pagini ascunse și directoare sensibile pe server.

Account Enumeration este vulnerabilitatea prin care o aplicație dezvăluie dacă un utilizator există în baza de date prin mesaje de eroare diferite (ex: "parolă greșită", "utilizator inexistent"). Atacatorii folosesc aceste indicii pentru a confirma liste de email-uri valide în vederea unor atacuri ulterioare de tip brute force. Protecția constă în returnarea unui mesaj generic și identic pentru orice eroare de autentificare, asigurând astfel că un atacator nu poate distinge între un cont real și unul fictiv.

5 Demonstare atac (PoC)

6 Analiză impact

7 Concluzii